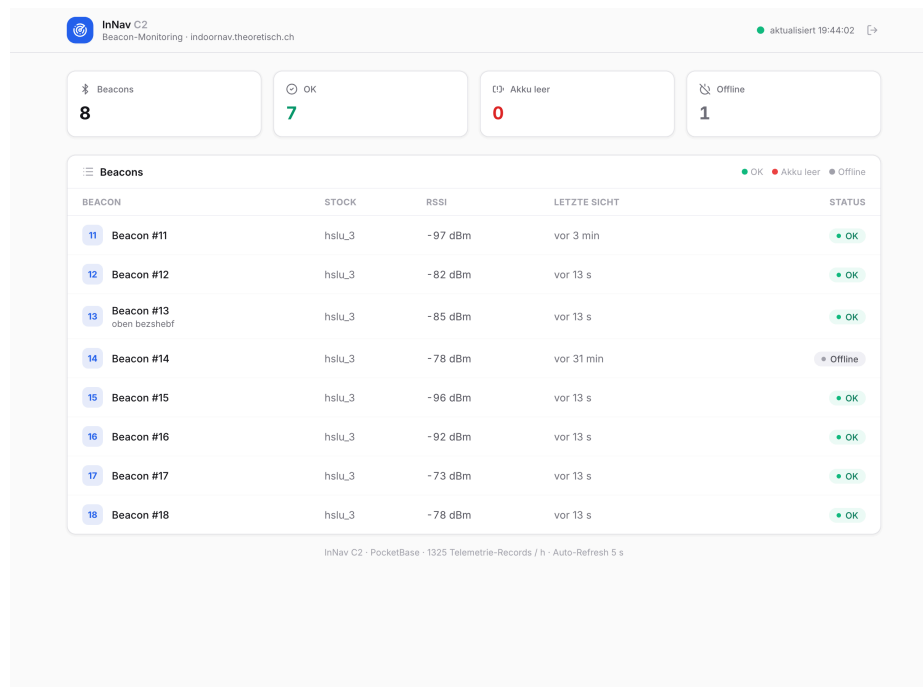


IIP2 — Indoor-Navigation HSLU Rotkreuz

Indoor-Navigationssystem auf dem Campus Rotkreuz



C2-Admin-Dashboard des Indoor-Navigationssystems

Modul: IIP2, Integrales Informatik Praxisprojekt 2

Semester: FS 2026

ECTS: 6 (Einzelarbeit)

Student: Theo Renz · theo.renz@stud.hslu.ch

Coach: Arnold Dieter & Jordan Sucur

Version: v1.0

Datum: 29.05.2026

I. Management Summary

Indoor-Lokalisierung ist auf dem HSLU-Campus Rotkreuz für Besucher und neue Studierende eine wiederkehrende Schwierigkeit, da GPS in Gebäuden nicht zuverlässig funktioniert. Im Rahmen des Moduls IIP2 wurde ein Prototyp eines BLE-basierten Indoor-Navigationssystems entwickelt, bestehend aus selbst gebauten ESP32-Beacons (XIAO ESP32 C6, iBeacon-Format mit eigener Akku-Telemetrie), einer Flutter-Mobile-App mit Live-Positionsbestimmung über lineare Trilateration und alternativ RSSI-Fingerprinting, sowie einem implementierten C2-Server (PocketBase, live unter indoornav.theoretisch.ch) für Konfigurations-Sync und Telemetrie. Die zentralen Algorithmen (Pfadverlust-Distanzschätzung, Trilateration und kNN-Fingerprinting) sind über 79 Unit-Tests validiert; eine standortbezogene Mess-Validierung erfolgt am HSLU-Campus. Aus dem Vergleich mit kommerziellen Lösungen wird abgeleitet, dass die selbstentwickelte, transparente Architektur bei deutlich geringeren Hardware-Kosten (< 6 CHF pro Beacon) und ohne Cloud-Abhängigkeit ein tragfähiger Lösungsweg für Hochschul-, Spital- und Museumsumgebungen ist.

II. Inhaltsverzeichnis

1. Problem, Fragestellung, Vision
 2. Stand der Praxis/Technik
 3. Ideen und Konzepte
 4. Methoden
 5. Realisierung
 6. Evaluation und Validation
 7. Ausblick
 8. Anhänge
 9. Abkürzungs-, Abbildungs-, Tabellenverzeichnis
 10. Literaturverzeichnis
 11. Deklaration für den Gebrauch von künstlicher Intelligenz
-

1. Problem, Fragestellung, Vision

GPS-basierte Ortung wird durch Dächer und Stahlbeton stark gedämpft und liefert in geschlossenen Gebäuden Genauigkeiten schlechter als 50 m (Zhao, 2018). Indoor-Lokalisierung als technische Disziplin behandelt die Bestimmung der Position innerhalb eines Gebäudes mit alternativen Funk- oder Sensor-Verfahren und wird in Krankenhäusern, Museen, Flughäfen und Hochschulen eingesetzt (Mautz, 2012).

Auf dem HSLU-Campus Rotkreuz haben Besucher des Info-Tags, neue Studierende und externe Gäste gemäss informellen Beobachtungen aus den Coachings I bis IV Schwierigkeiten, einzelne Räume zu finden. Die Aufgabenstellung des Moduls IIP2 (Hochschule Luzern, 2026, Anhang A) verlangt einen funktionsfähigen Prototyp eines Indoor-Navigationssystems für einen beispielhaften Anwendungsfall. Als Anwendungsfall wurde der HSLU-Campus Rotkreuz, Gebäude Suurstoffi, Stockwerk 3 gewählt.

Ziele dieser Arbeit:

- **Ziel 1, Technologie-Eignung:** Beurteilung, mit welcher Genauigkeit ein Bluetooth-Low-Energy-basiertes Indoor-Lokalisierungssystem auf dem HSLU-Campus bei einer minimalen Beacon-Dichte von drei bis acht Beacons pro Stockwerk arbeitet.
- **Ziel 2, Architektur-Wahl:** Erarbeitung einer Verantwortungs-Aufteilung zwischen Hardware-Beacons, Mobile-App und einem zentralen Backend, welche skalierbar bleibt und nach der Erstinstallation ohne Server-Erreichbarkeit funktioniert.
- **Ziel 3, Lauffähiger Prototyp:** Implementation einer Mobile-Anwendung, welche die BLE-Signale empfängt, die Position auf einem digitalen Stockwerksplan berechnet und für eine Live-Demonstration am IIP2Fair (02.06.2026) und an der Schlusspräsentation (03.07.2026) bereitsteht.

Tabelle 1: Ziele und erwartete Outputs (eigene Darstellung)

Nr.	Ziel	Output
1	Technologie-Eignung BLE	Vergleichstabelle BLE gegenüber UWB, WLAN-RTT und visuellen Markern (Kapitel 2.1)

Nr.	Ziel	Output
2	Architektur-Wahl	Drei-Schicht-Architektur Beacons → Mobile-App → C2-Server (Kapitel 3.1 und 5)
3	Lauffähiger Prototyp	Flutter-App mit Live-Trilateration + Fingerprinting, Quellcode-Repository (Kapitel 5 und 6)

2. Stand der Praxis/Technik

2.1 Indoor-Lokalisierungs-Technologien

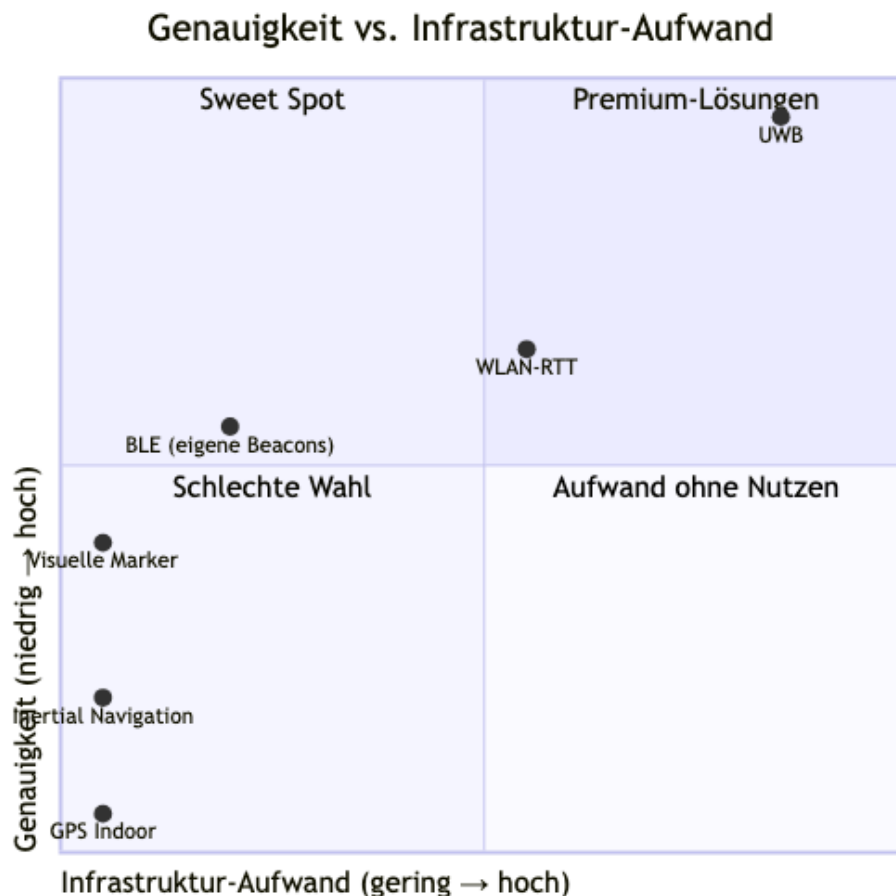
Mautz (2012) klassifiziert die für Indoor-Lokalisierung verfügbaren Technologien nach ihrem physikalischen Trägermedium in fünf Hauptgruppen. Tabelle 2 fasst die für ein Hochschul-Setting relevanten Eigenschaften zusammen.

Tabelle 2: Indoor-Lokalisierungs-Technologien im Vergleich

Technologie	Typische Genauigkeit	Infrastruktur-Aufwand	Smartphone-Verfügbarkeit	Quelle
GPS (Outdoor-Referenz)	3–10 m im Freien, > 50 m indoor	k. A.	nativ	Zhao (2018)
BLE (Bluetooth Low Energy)	1–5 m	gering: 3–10 Beacons / 100 m ²	nativ ab iOS 7 und Android 4.3	Mustafa und Sykes (2021)
UWB (Ultra-Wideband)	0.1–0.5 m	hoch: Anker und spezielle Chips	nur iPhone 11+ und einige Android-Flaggschiffe	Ponte et al. (2024)
WLAN-RTT (Fine Time Measurement)	1–2 m	mittel: kompatible Access-Points	nur Android 9+, kein iOS	Mautz (2012)
Visuelle Marker (QR, ArUco)	0.5–1 m am Marker	sehr gering	nativ	Mautz (2012)
Inertial Navigation (IMU)	Drift 1–3 m / Minute	keine	nativ	Ponte et al. (2024)

Aus Tabelle 2 wird abgeleitet, dass für den vorliegenden Anwendungsfall (Hochschul-Campus, beide Smartphone-Plattformen, geringe Infrastruktur-Investition, Genauigkeit im Meter-Bereich ausreichend) BLE die einzige Technologie ist, welche alle Anforderungen ohne Kompromiss erfüllt. UWB scheidet wegen der iPhone-Verfügbarkeit unter Modell 11 aus, WLAN-RTT wegen fehlender iOS-Unterstützung. Visuelle Marker und IMU sind als Ergänzung möglich, aber nicht als alleinige Lösung tragfähig.

Abbildung 1: Indoor-Lokalisierungs-Technologien, Genauigkeit gegenüber Infrastruktur-Aufwand



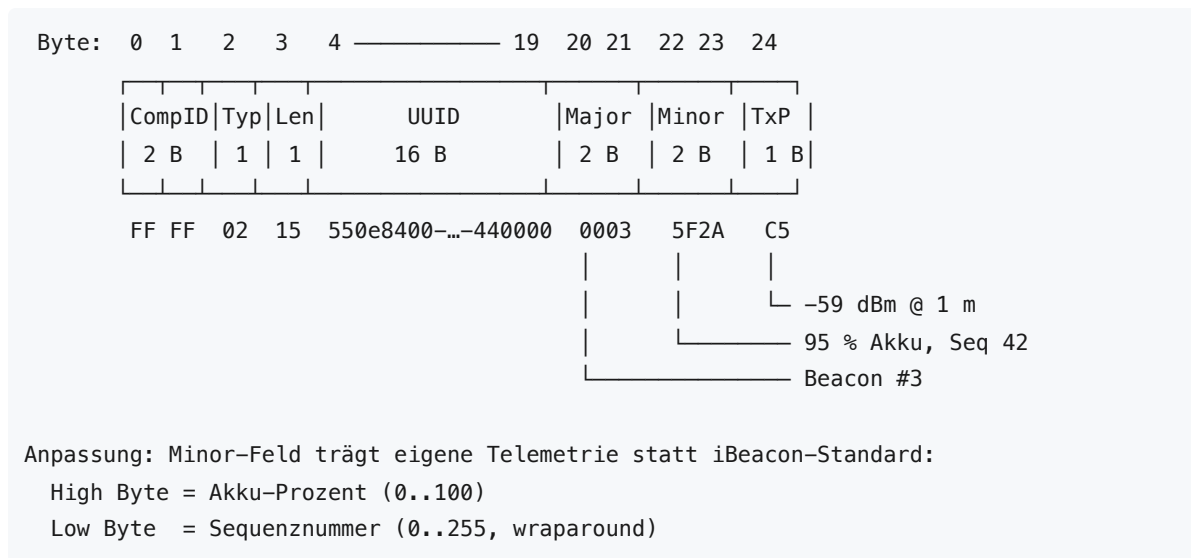
Diagramm

2.2 BLE-Beacons und iBeacon-Protokoll

Bluetooth Low Energy ist ein 2007 standardisiertes Funkprotokoll für stromsparende Datenübertragung im 2.4-GHz-ISM-Band (Bluetooth SIG, 2019). Ein Beacon im Sinne dieser Arbeit ist ein BLE-Sender, der periodisch ein Advertisement-Paket aussendet, ohne eine Verbindung aufzubauen. Die Sendefrequenz liegt typischerweise zwischen 100 ms und 2'000 ms.

Apple (2014) hat mit dem **iBeacon-Format** einen De-facto-Standard für Indoor-Lokalisierungs-Beacons geschaffen. Ein iBeacon-Paket trägt eine 128-Bit-UUID zur Identifikation des Sender-Systems, ein 16-Bit-Major-Feld zur Gerätenummerierung, ein 16-Bit-Minor-Feld zur Sub-Identifikation und ein 8-Bit-TX-Power-Feld, welches die bei einem Meter Distanz gemessene Signalstärke trägt (siehe Tabelle 3 in Kapitel 5.2 für die vorliegende Implementation). Das Format ist herstelleroffen, sodass auch Drittanbieter-Beacons und selbstgebaute ESP32-basierte Sender empfangen werden können.

Abbildung 2: iBeacon-Paketstruktur (25 Bytes Manufacturer-Data, eigene Telemetrie in Minor)



2.3 RSSI-basierte Distanzberechnung

Der Received Signal Strength Indicator (RSSI) ist die vom Empfänger gemessene Signalstärke in dBm. Das logarithmische Pfadverlust-Modell beschreibt den Zusammenhang zwischen RSSI und Distanz nach der Formel

$$d = 10^{\left(\frac{TxPower - RSSI}{10 \cdot n} \right)} \text{ (Formel 1)}$$

wobei n den umgebungs-spezifischen Pfadverlust-Exponenten bezeichnet (Zhao, 2018). Für offene Hallen liegt n nahe bei 2.0 (Freifeld-Annäherung), in Büro-Umgebungen mit Wänden und Möblierung zwischen 2.5 und 3.5. Der Wert ist nicht konstant und muss pro Umgebung kalibriert werden. Die vorliegende Implementation verwendet $n = 2.5$ als Indoor-Default (Code-Referenz: `code/mobileapp/lib/models/beacon.dart, distanceFromRssi`).

In der Praxis schwanken die RSSI-Werte um ± 5 dB durch Multipath-Reflexionen, Körper-Dämpfung des Nutzers und Antennen-Orientierung des Empfänger-Smartphones. Mustafa und Sykes (2021) zeigen, dass ohne Glättung die abgeleitete Distanz um ± 1.5 m schwankt, selbst bei statischer Position. Diese Werte gelten bei dichter Beacon-Abdeckung; mit weniger Beacons fällt die Genauigkeit erfahrungsgemäss geringer aus.

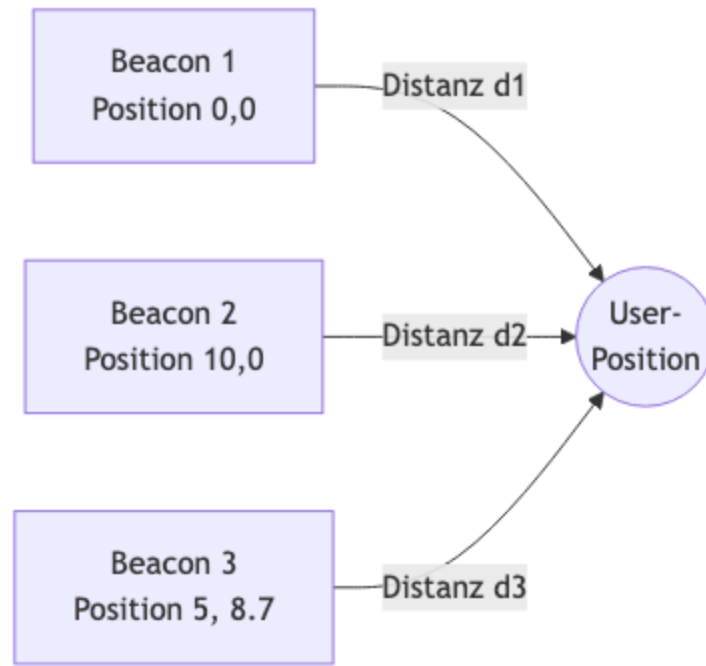
2.4 Lokalisierungs-Verfahren mit BLE

Innerhalb der BLE-basierten Verfahren werden in der Literatur drei Hauptansätze unterschieden (Zhao, 2018):

Proximity-Detektion bestimmt nur, welcher Beacon das stärkste Signal sendet, und ordnet den Nutzer dem zugehörigen Raum zu. Vorteil ist die einfache Implementation, Nachteil ist die niedrige Auflösung im Bereich der Beacon-Reichweite (typisch 5 bis 10 m).

Trilateration schätzt die Position aus den Abständen zu mindestens drei Beacons mit bekannten Positionen. Jeder Beacon liefert über seine Signalstärke einen ungefähren Abstand; der gesuchte Standort ist der Punkt, der am besten zu allen Abständen passt. Foy (1976) beschreibt dafür ein Verfahren, das diese Berechnung als einfaches lineares Gleichungssystem schnell löst.

Abbildung 3: Trilateration, drei Beacons mit bekannter Position bestimmen die User-Position



Diagramm

Fingerprinting kommt ohne Distanzformel aus. Zuerst werden an bekannten Punkten die Signalstärken aller Beacons gemessen und gespeichert. Im Betrieb vergleicht die App die aktuelle Messung mit diesen Mustern und wählt die am besten passende Position. Mustafa und Sykes (2021) erreichen damit in einem Bürogebäude rund 2.2 m Genauigkeit, etwas besser als reine Trilateration. Beide Verfahren werden in dieser Arbeit umgesetzt und in Kapitel 6 verglichen.

2.5 Existierende Lösungen

Google Maps Indoor und **Apple Indoor Maps** decken einzelne Flughäfen, Bahnhöfe und Einkaufszentren ab. Eine Aufnahme in diese Karten erfordert eine kommerzielle Vereinbarung mit dem Plattform-Betreiber und ist für einzelne Hochschul-Gebäude nicht vorgesehen (Apple Inc., 2024). Beide Lösungen kapseln den Algorithmus und liefern dem Entwickler keine Roh-RSSI-Werte, was eine wissenschaftliche Evaluation der zugrunde liegenden Verfahren verhindert.

HERE Indoor Radio Positioning und **IndoorAtlas** sind kommerzielle Plattformen, welche eigene Beacon-Hardware oder eine Geo-Magnetfeld-Karte voraussetzen. Beide sind für Pilot-Projekte mit Lizenzkosten verbunden.

Tabelle 3 fasst die Differenzierung der vorliegenden Arbeit zu den existierenden Lösungen zusammen.

Tabelle 3: Differenzierung zur eigenen Arbeit

Kriterium	Bestehende kommerzielle Lösungen	Vorliegende Arbeit
Skalierung	gebäude-spezifisch, kommerziell	Open-Source, Stockwerk-konfigurierbar
Cloud-Abhängigkeit	Pflicht (Google, Apple, HERE)	Offline-fähig nach Erstinstallation
Hardware-Kosten	30–80 CHF pro Beacon	< 10 CHF pro Beacon (XIAO ESP32 C6)
Methoden-Transparenz	Black Box	vollständig dokumentiert, beide Methoden (Trilateration + Fingerprinting) implementiert
Geräte-Voraussetzung	iOS- oder Android-spezifisch oder UWB-Chip	iOS 13+ und Android 4.3+ (BLE 4.0)

3. Ideen und Konzepte

3.1 Lösungsarchitektur

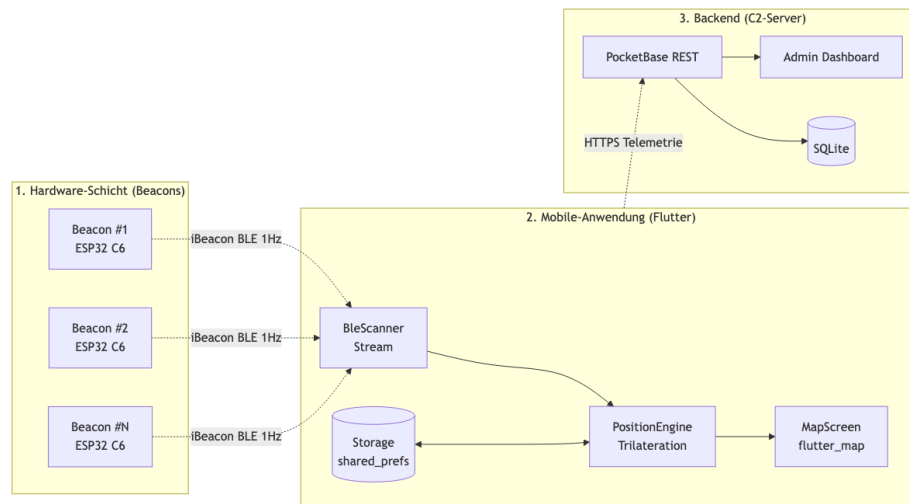
Aus den in Kapitel 2.4 verglichenen Verfahren wird für die vorliegende Arbeit eine **drei-schichtige Architektur** abgeleitet:

1. **Hardware-Schicht:** ESP32-basierte BLE-Beacons, welche periodisch iBeacon-Pakete senden. Ein Beacon ist bewusst als reiner Sender ausgelegt; eine Verbindung zum Empfänger wird nicht aufgebaut, um den Stromverbrauch unter 1 mA Durchschnitt zu halten und damit eine Akku-Laufzeit von Wochen bis Monaten zu erreichen.
2. **Mobile-Anwendungs-Schicht:** Eine Flutter-Anwendung empfängt die BLE-Advertisements, glättet die RSSI-Werte über einen Moving-Average-Filter, berechnet die Position auf dem Stockwerksplan und zeichnet sie als Marker. Die Anwendung ist offline-fähig nach der Erstinstallation.
3. **Backend-Schicht (C2-Server):** Ein zentraler Dienst verwaltet die Beacon-Konfiguration (Position, TX-Power, Label), sammelt Telemetrie-Daten von den Mobile-Geräten und stellt einem Administrator ein Dashboard zur Akku-

Überwachung bereit. Der Server ist implementiert (PocketBase) und unter indoornav.theoretisch.ch deployed (Details in Kapitel 5.6).

Diese Verteilung folgt dem Grundsatz, dass die Position **lokal auf dem Endgerät** berechnet wird, sodass keine Server-Erreichbarkeit für die Kernfunktion nötig ist.

Abbildung 4: System-Architektur, drei Schichten mit Datenfluss



Diagramm

Die Position wird auf dem Endgerät berechnet; der C2-Server (Kapitel 5.6) übernimmt Konfigurations-Sync, Telemetrie-Sammlung und das Admin-Dashboard, ist für die Kern-Lokalisierung aber nicht erforderlich.

3.2 Konzept-Vergleich der Lokalisierungs-Methoden

Aus dem Stand der Technik (Kapitel 2.4) werden zwei Verfahren zur Implementation ausgewählt: **Trilateration** als geometrisch-deterministisches Verfahren und **Fingerprinting** als datenbasiertes Verfahren. Aus der Literatur (Mustafa und Sykes, 2021) wird abgeleitet, dass beide Verfahren komplementäre Stärken haben (Trilateration ohne Trainingsaufwand, Fingerprinting mit höherer Genauigkeit nach Kalibrierung) und ein Vergleich beider in Kapitel 6 sinnvoll ist.

3.3 Scope und Abgrenzung

Tabelle 4: Scope-Definition Phase 1 (eigene Darstellung)

Im Scope (Phase 1, Abgabe 29.05.2026)	Out of Scope (Phase 2, vor 03.07.2026)	Out of Scope (Phase 3+, Vision)
BLE-Scanner mit iBeacon-Parser	Kalibrierungs-Seite mit Fingerprint-Capture	Sensor-Fusion mit IMU (Dead Reckoning)
Pfadverlust-Distanzschätzung	C2-Server (PocketBase + Admin-Dashboard)	Echter 2D-Kalman-Filter
Trilateration via linearer Least-Squares	Fingerprinting kNN-Matcher	A*-Routing auf Gebäude-Graph
Vektor-Floor-Plan (HSLU-Stock 3)	App↔C2 Telemetrie-Upload	Turn-by-Turn-Navigation
Setup-Screen (Beacon-Platzierung)	Multi-Floor-Wechsler-UX	HSLU-App-Integration
Map-Screen mit Live-Position	Apple TestFlight Distribution	Fingerprint-Crowdsourcing

Der Phase-1-Scope wurde bewusst klein gehalten (Coaching IV). Mehrere ursprünglich als spätere Erweiterung geplante Module sind inzwischen trotzdem umgesetzt: C2-Server, Fingerprinting sowie App-C2-Telemetrie und -Sync (siehe Kapitel 5). Offen bleiben die standortbezogene Mess-Validierung, die TX-Power-Kalibrierung und die Multi-Floor-UX.

4. Methoden

4.1 Vorgehensmodell

Die Bearbeitung erfolgte über das ganze Semester (Semesterwochen 1 bis 14) iterativ entlang des Coaching-Rhythmus (Coachings I bis IV). Hardware-Beschaffung, Firmware-Entwicklung, Mobile-App-Implementation und Dokumentation liefen parallel; vor jedem Coaching wurde ein vorzeigbarer Zwischenstand erzeugt und in einem Arbeitsjournal (Anhang D) dokumentiert.

4.2 Wissenschaftliche Methoden

Da es sich um eine Einzelarbeit mit Schwerpunkt auf der Umsetzung handelt, kamen zwei Methoden zum Einsatz. Eine **vergleichende Literatur-Analyse** der Indoor-Lokalisierungs-Technologien (Kapitel 2.1) auf Basis von Mautz (2012) und

Zhao (2018) begründet die Technologie-Wahl BLE. Eine **Funktions-Validierung am Standort** prüft, ob die App die Beacons empfängt und eine plausible Live-Position auf dem Grundriss anzeigt (Kapitel 6).

4.3 Hardware-Plattform

Als Beacon-Hardware wurde der **Seeed Studio XIAO ESP32 C6** ausgewählt.
Begründung der Wahl in tabellarischer Form:

Tabelle 5: Auswahlkriterien Beacon-Hardware (eigene Darstellung)

Kriterium	XIAO ESP32 C6	Alternative: nRF52840	Alternative: Estimote LT
BLE 5.0	✓	✓	✓
Stromverbrauch Deep Sleep	7 µA	1.5 µA	k. A.
Akku-Lade-IC integriert	ETA4054	k. A.	k. A.
Stückpreis (CHF)	4.53	22.00	ca. 30
Verfügbarkeit Mouser CH	✓	✓	nur direkt
Dokumentation Open Source	ausführlich	ausführlich	proprietär

Aus Tabelle 5 ergibt sich der ESP32 C6 als beste Wahl für das verfügbare Budget. Die Hardware für 20 Beacons kostete rund 156 CHF: 20 Boards für 93.15 CHF (3 Kits à 5.38 CHF, 17 Boards à 4.53 CHF) und 8 Akkus für 63.20 CHF (je 7.90 CHF).

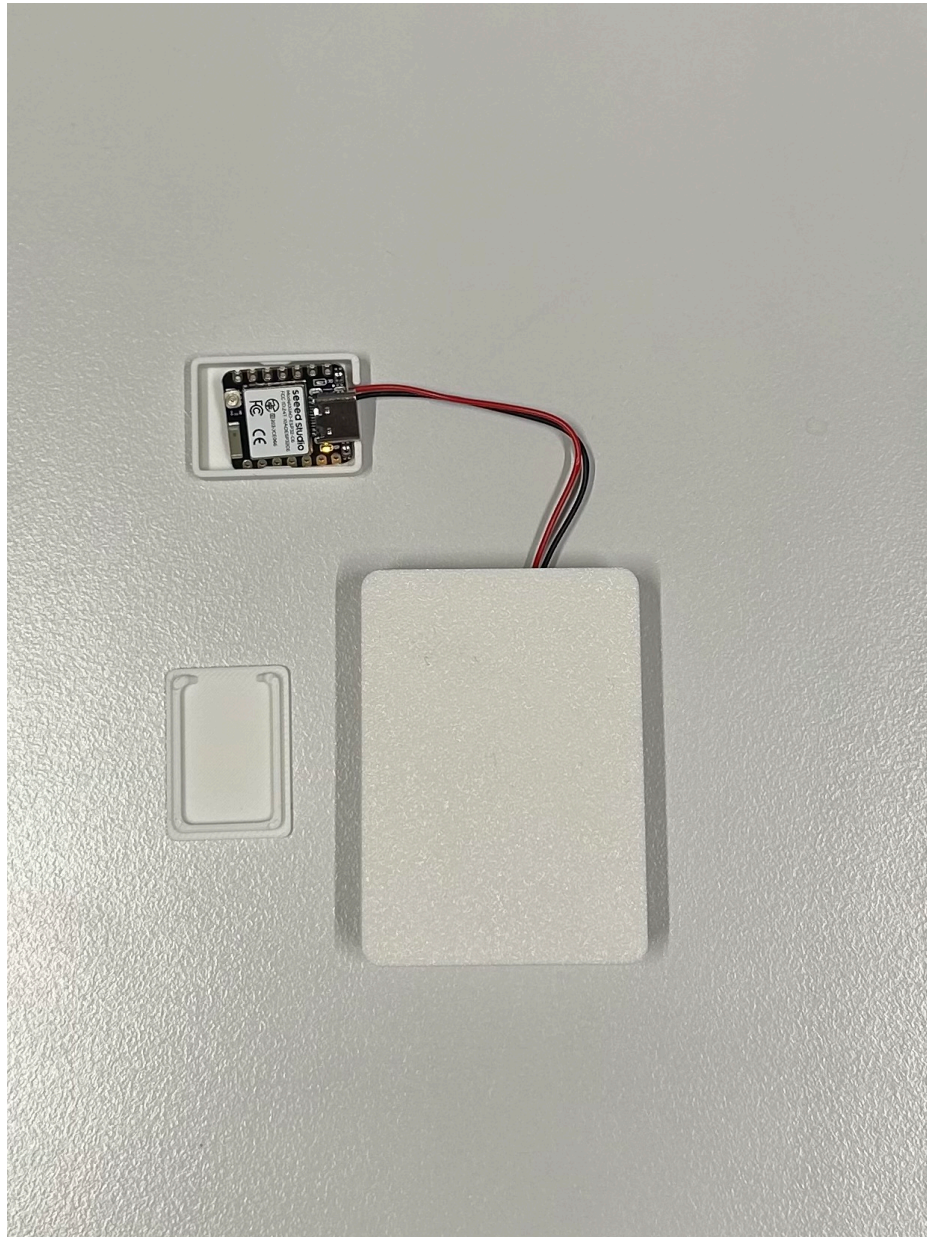
4.4 Software-Stack

Für die Mobile-App wurden **Flutter und Dart** gewählt, weil sie plattformübergreifend für iOS und Android funktionieren und damit nur eine Code-Basis nötig ist. Die BLE-Kommunikation übernimmt das Plugin **flutter_blue_plus**, die Beacon-Firmware entsteht mit **PlatformIO** (Arduino-ESP32, NimBLE). Der C2-Server nutzt **PocketBase**, gewählt aus Gründen der Einfachheit, da Backend, Datenbank und Admin-Oberfläche in einem Werkzeug enthalten sind.

5. Realisierung

5.1 Hardware-Aufbau Beacon

Abbildung 5: Beacon-Hardware, XIAO ESP32 C6 mit LiPo-Akku im 3D-gedruckten Gehäuse



Beacon-Hardware: XIAO ESP32 C6 mit Akku im 3D-gedruckten Gehäuse

XIAO ESP32 C6 mit angelötetem LiPo-Akku, gefasst im 3D-gedruckten Gehäuse (eigene Aufnahme).

Pro Beacon werden ein **Seeed XIAO ESP32 C6** (4.53 CHF), ein **Akyga LP803040 LiPo-Akku** (1000 mAh, 7.90 CHF) und ein selbst gedrucktes 3D-Gehäuse (Materialkosten unter 1 CHF pro Stück) kombiniert. Der Akku wird direkt an die BAT+/BAT--Pads des ESP32-Boards gelötet; eine JST-Buchse ist auf dem XIAO nicht verfügbar (Seeed Studio, 2024). Das Laden über USB-C funktioniert dank des integrierten ETA4054-Lade-ICs auch im verbauten Zustand.

Die Beacon-Montage erfolgt mit Tesa Powerbond Ultra Strong an Wänden oder Türrahmen in einer Höhe von 2.0 bis 2.5 m. Die empfohlene Beacon-Dichte für die in Kapitel 6 angestrebte Genauigkeit beträgt drei Beacons pro Raum mit triangularer Anordnung und einem mittleren Abstand von 4 bis 6 m. Diese Werte sind aus Zhao (2018) abgeleitet.

5.2 Beacon-Firmware

Die Firmware ist in C++ mit dem Arduino-ESP32-Framework (Version 3.x, NimBLE-Stack) implementiert und in der Datei `code/beacons/firmware/src/main.cpp` abgelegt (585 Code-Zeilen). Sie sendet alle 1'000 ms ein iBeacon-Advertisement-Paket nach Tabelle 3.

Tabelle 3: iBeacon-Paketformat (InNav v1) (eigene Darstellung)

Feld	Bytes	Bedeutung
Company ID	2	0xFFFF (unassigned / test)
iBeacon-Typ	2	0x02 0x15
UUID	16	InNav-Firmware-Version
Major	2	Beacon-ID (Build-Flag)
Minor (High)	1	Akkustand 0–100 %
Minor (Low)	1	Sequenznummer 0–255
TX Power	1	kalibrierter RSSI bei 1 m

Die Beacon-spezifische Konfiguration (Beacon-ID, kalibrierte TX-Power, Advertisement-Intervall) wird über Build-Flags in `platformio.ini` gesetzt, sodass dieselbe Firmware ohne Code-Änderung für mehrere Beacons gebaut werden kann. Eine selbst erstellte **LiPo-Entlade-Lookup-Tabelle** mit elf Stützpunkten (4.2 V bis 3.0 V) schätzt die nichtlineare Entladekurve eines LiPo-Akkus und ist genauer als eine lineare $V \rightarrow \%$ -Umrechnung. Der **Watchdog-Timer** (30 Sekunden) erkennt Software-Hänger und startet den Beacon automatisch neu. Im aktivierten Deep-

Sleep-Modus zwischen den Advertisements beträgt der Stromverbrauch laut Datenblatt 7 μA (Espressif Systems, 2024), was bei einem 1'000 mAh-Akku theoretisch eine Laufzeit von 40 bis 80 Tagen ergibt.

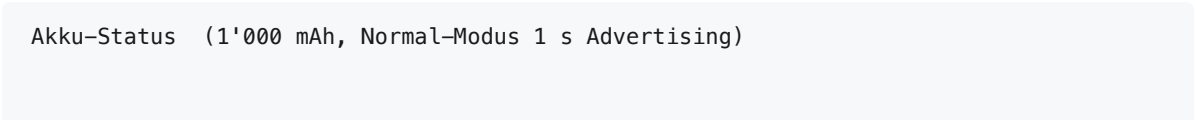
Akku-Status-Telemetrie und Energiekonzept. Da der XIAO ESP32 C6 die rohe Akku-Spannung ohne externen Spannungsteiler nicht über den ADC zugänglich macht (siehe ADR-007 und Kapitel 5.8), überträgt die Firmware im Minor-High-Byte ein **binäres Status-Flag** statt eines unzuverlässigen Prozentwerts: 100 (OK) im Normalbetrieb, 5 (kritisch) nach einem Brownout-Reset. Der Brownout-Detektor des ESP32 misst die Versorgungsspannung direkt und löst bei Unterschreiten der Entladeschlussspannung (rund 2.9 V) aus. In diesem Fall wechselt der Beacon in einen **Schon-Modus** mit verlängertem Advertisement-Intervall (60 s statt 1 s), wodurch sich der Durchschnittsstrom um etwa Faktor 45 reduziert und das verbleibende Service-Fenster verlängert wird.

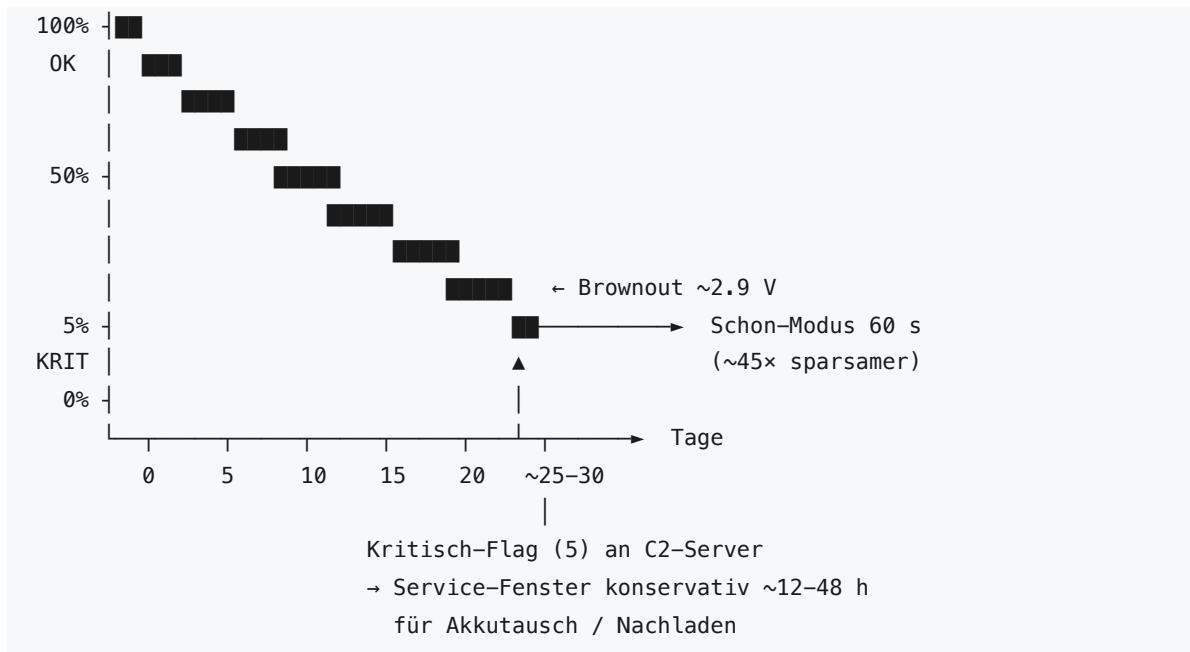
Bedienung und Statusanzeige. Eine orange Heartbeat-LED (GPIO15) pulst bei jedem Advertisement und zeigt ohne Smartphone, dass der Beacon sendet; die rote Lade-LED des ETA4054-ICs signalisiert den Ladevorgang. Der BOOT-Taster dient als Aus-Schalter: einige Sekunden gedrückt gehalten, versetzt er den Beacon in einen unbefristeten Deep-Sleep, der Wiederstart erfolgt über den RESET-Taster. Da GPIO9 (BOOT) auf dem ESP32-C6 nicht RTC-fähig ist und nicht aus dem Deep-Sleep wecken kann, wird der Taster in der Wach-Phase gepollt statt als GPIO-Wakeup genutzt (siehe ADR-008).

Tabelle 8: Berechnete Energiebilanz pro Betriebsmodus (1'000 mAh LiPo, $\approx$ 800 mAh nutzbar) (eigene Darstellung)

Modus	Intervall	$\varnothing$ -Strom (berechnet)	Laufzeit
Normal (Advertising + Heartbeat-LED)	1 s	$\sim 1.4\text{ mA}$	$\sim 24\text{--}30\text{ Tage}$
Normal ohne Heartbeat-LED	1 s	$\sim 1.0\text{ mA}$	$\sim 33\text{--}40\text{ Tage}$
Brownout-Schon-Modus	60 s	$\sim 30\text{ }\mu\text{A}$	Restkapazität-limitiert

Abbildung 6: Geschätzte Akku-Entladekurve und Service-Fenster nach Kritisch-Meldung



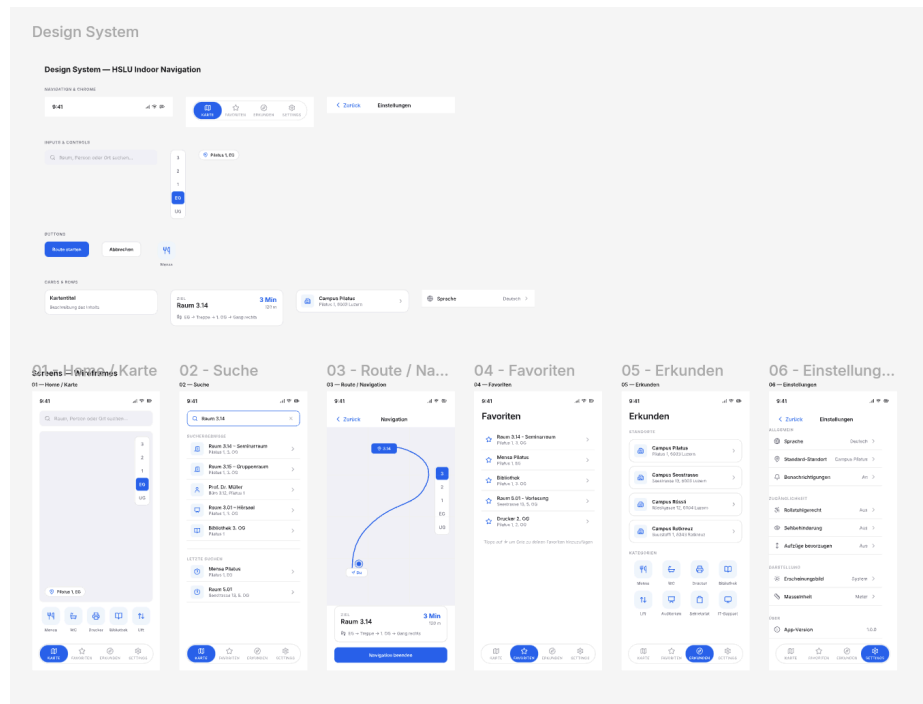


Aus dem binären Flag wird ein **Wartungs-Workflow** für den C2-Server abgeleitet: Sobald ein Beacon das Kritisch-Flag (5) sendet, markiert das Admin-Dashboard ihn farblich und kann dem Servicepersonal ein verbleibendes Zeitfenster anzeigen (z. B. „Beacon 14 kritisch, ca. 12 h bis voraussichtlicher Ausfall“). Die genannten Laufzeit-Werte sind aus den Datenblatt-Strömen und dem Tastverhältnis **berechnet**; eine reale Entladungs-Messreihe zur Validierung des Service-Fensters ist als Phase-2-Aufgabe vorgesehen (Kapitel 7.2).

5.3 Mobile-App-Architektur

Vor der Implementierung entstand ein erster UI-Entwurf in Pencil.dev (Abbildung 7), der die spätere Umsetzung leitete.

Abbildung 7: Erster UI-Entwurf der Mobile-App (Pencil.dev, vor der Implementierung)



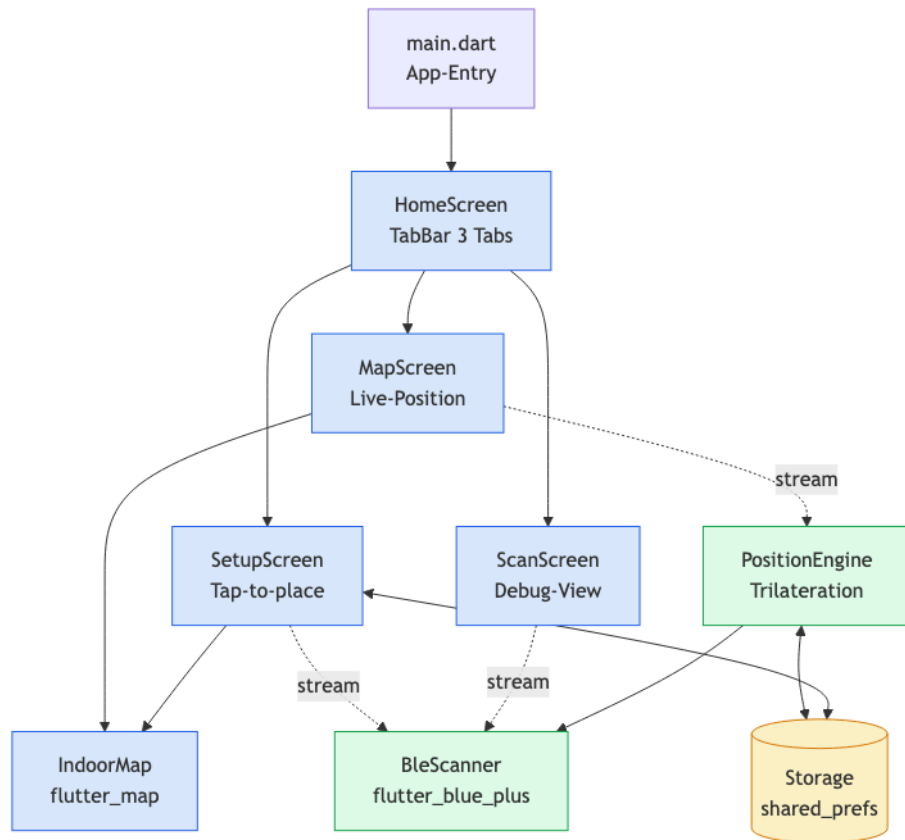
Erster UI-Entwurf in Pencil.dev

Erster Design-Entwurf in Pencil.dev vor dem Coden (eigene Darstellung).

Die Flutter-Anwendung ist in vier Schichten gegliedert (Verzeichnis `code/mobileapp/lib/`):

- **models/**: BeaconScan, BeaconPlacement, FloorPlan, PositionEstimate, Fingerprint, WalkableZone, TelemetryRecord, PositionStrategy: immutable Datenklassen mit JSON-Serialisierung
- **services/**: BleScanner, Storage, PositionEngine, ApiService, SyncService, TelemetryUploader, walkable: Singletons mit Stream-API
- **screens/**: HomeScreen (Besucher-Modus: nur Karte; Admin-Modus: vier Tabs), MapScreen, SetupScreen, ScanScreen, CalibrationScreen, WalkableScreen, SettingsScreen
- **widgets/**: IndoorMap (flutter_map-Renderer mit CrsSimple-Projektion)

Abbildung 8: Mobile-App-Modul-Struktur (`code/mobileapp/lib/`)



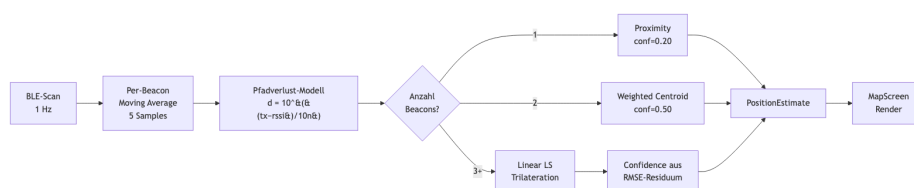
Diagramm

Als **State-Management** wurde bewusst auf eine externe Bibliothek wie Provider oder Riverpod verzichtet (siehe ADR-001 in Anhang B). Stattdessen halten die Singleton-Services einen Broadcast-Stream, den die Screens via StreamBuilder konsumieren. Diese Wahl reduziert externe Abhängigkeiten, vereinfacht Tests (Services direkt instanzierbar) und ist für die drei aktuellen Screens ausreichend.

Der BleScanner umschließt die flutter_blue_plus-Bibliothek (flutter_blue_plus Contributors, 2026, Version 1.35.2), parst iBeacon-Advertisements und filtert auf die eigene UUID. Empfangene Pakete werden als BeaconScan-Objekte mit Beacon-ID, Akku-Prozent, Sequenznummer, RSSI und TX-Power publiziert.

5.4 Position-Engine und Trilateration

Abbildung 9: Position-Engine-Pipeline, RSSI-Stream zu PositionEstimate



Diagramm

Die zentrale Algorithmus-Komponente `PositionEngine` (`code/mobileapp/lib/services/position_engine.dart`, etwa 250 Code-Zeilen) realisiert diese drei-stufige Pipeline:

1. **RSSI-Glättung** per gleitendem Mittelwert über die letzten fünf Samples pro Beacon. Aus den in Kapitel 2.3 zitierten ± 5 dB Multipath-Schwankungen folgt die Notwendigkeit dieser Glättung; ein vollständiger Kalman-Filter wird in Kapitel 7 als Phase-2-Ausblick formuliert (siehe ADR-003 in Anhang B).
2. **Distanz-Schätzung** per Pfadverlust-Modell (Formel 1) mit Exponent $n = 2.5$ als Indoor-Default. Pro Beacon wird der TX-Power-Override aus der `BeaconPlacement` verwendet, falls vorhanden; sonst der vom Beacon selbst gesendete Default-Wert.
3. **Position-Berechnung** abhängig von der Anzahl sichtbarer Beacons:
 - **1 Beacon** → **Proximity**: Position = Beacon-Position, Confidence 0.20
 - **2 Beacons** → **Weighted Centroid**: Position als $1/d$ -gewichteter Mittelwert auf der Verbindungslinie, Confidence 0.50
 - **3+ Beacons** → **Trilateration**: Linearisierte Methode nach Foy (1976), mit Referenz-Beacon-Subtraktion und Cramer-Regel für das resultierende 2×2 -Normalengleichungs-System. Confidence aus dem Wurzel-mittleren-Quadrat-Fehler der Residuen (RMSE)

Die Wahl der linearisierten Methode gegenüber einem iterativen Levenberg-Marquardt-Verfahren ist in ADR-006 (Anhang B) begründet: Bei 1–2 Hz Update-Rate ist die <1 ms-Lösung des linearen Verfahrens ausreichend, und die marginal höhere Genauigkeit der iterativen Methoden wird durch das RSSI-Glättungs-Vorverfahren weitgehend kompensiert.

Liegen alle drei Beacons (fast) auf einer Linie, lässt sich die Position mathematisch nicht eindeutig bestimmen; in diesem Fall zeigt die App „keine Position berechenbar“ an.

Über die geometrische Trilateration hinaus implementiert die `PositionEngine` zwei weitere Verfahren, zwischen denen in den Einstellungen gewählt wird:

Fingerprinting vergleicht die aktuelle RSSI-Signatur über einen k -Nearest-Neighbors-Abgleich gegen zuvor erfasste Referenzpunkte, und ein **Hybrid-Modus** kombiniert Trilateration und Fingerprinting confidence-gewichtet. Liegen keine

Fingerprints vor, fällt der Hybrid-Modus automatisch auf reine Trilateration zurück. Die Referenzpunkte werden im **Kalibrierungs-Screen** (Admin-Modus) erfasst: Der Nutzer tippt seine Position auf der Karte an, die App mittelt über rund 60 Sekunden die RSSI-Werte aller sichtbaren Beacons und speichert das Ergebnis als Fingerprint. Mustafa und Sykes (2021) leiten aus Bürogebäude-Messungen für Fingerprinting eine mittlere Genauigkeit von 2.2 m ab; ein quantitativer Vergleich beider Verfahren am HSLU-Standort steht aus (Kapitel 6.3).

5.5 Floor-Plan-Modell und Map-Rendering

Das Stockwerk wird als Vektor-Modell FloorPlan repräsentiert (Datei `code/mobileapp/lib/models/floor_plan.dart`). Es enthält die Gebäude-Dimensionen in Metern, eine Liste von Room-Objekten (Rechtecke mit ID und Label) und eine Liste von PointOfInterest-Markern (Lift, Treppe, WC). Das Koordinaten-System ist in **Metern**, mit dem Ursprung in der oberen linken Ecke, diese Konvention spiegelt das übliche Screen-Layout und vermeidet einen Y-Achsen-Flip beim Rendern.

Für die Validation wurde das **HSLU-Suurstoffi-Stockwerk 3** als FloorPlan-Konstante modelliert. Das Layout wurde aus dem Architektur-Grundriss (Quelle: HSLU-Bauplan) abgeleitet, mit Raum 300 (Erschliessung, Bodenfläche 367.15 m²) als Maßstabs-Anker. Das Modell umfasst 18 Räume mit ihren offiziellen HSLU-Nummern (300 bis 337) und 8 POIs (zwei Treppenhäuser, zwei Lifte, zwei WCs, Wendeltreppe in der Mitte, Aufenthalts-Lounge). Gebäude-Dimensionen: 70 × 42 m, brutto etwa 2'940 m².

Abbildung 10: HSLU Suurstoffi, Stockwerk 3, Architektur-Grundriss (Quelle der Vermessung)



HSLU Suurstoffi Stockwerk 3 Grundriss

Architektur-Grundriss aus dem HSLU-Bauplan. Sichtbar: 18 Räume mit Nummern 300–337, zwei symmetrische Treppenhäuser (335 West, 315 Ost), Wendeltreppe in der Mitte des Korridors (Raum 300 „Erschliessung“, BF 367.15 m²), WCs (337 ♀, 317 ♂) und Service-Räume (Netzwerkraum 316, Starkstrom 336). Aus diesem Plan wurden die Koordinaten der FloorPlan-Konstante in `code/mobileapp/lib/models/floor_plan.dart` abgeleitet.

Das IndoorMap-Widget basiert auf der Bibliothek `flutter_map` mit einer `CrsSimple`-Projektion (kartesisches Meter-Koordinatensystem statt Geo-Koordinaten; Datei `code/mobileapp/lib/widgets/indoor_map.dart`). Dies ersetzt den ursprünglichen `CustomPaint`-Renderer und liefert Pan-, Pinch-Zoom- und Rotations-Gesten ohne Eigenentwicklung. Gezeichnet werden als Layer: optionaler Grundriss als Hintergrundbild, Raum-Marker, begehbare und nicht-begehbare Zonen, RSSI-Distanzringe, Beacon-Marker und der User-Position-Marker mit Confidence-Pulse.

Abbildung 11: Mobile-App, Map-Screen mit Live-Position (Stockwerk 3)

19:41

📶 100

🏠 HSLU Suurstoffi, Stock 3 ▾



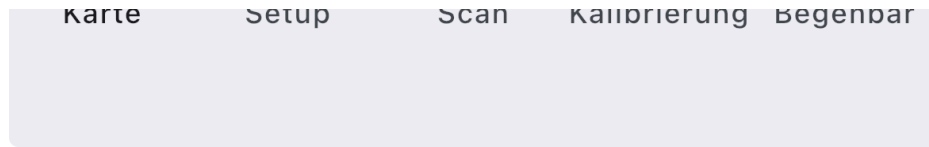
33.95 m, 28.57 m

Trilateration · 5 Beacons · RMSE 10.96 m

Confidence

0%





Mobile-App Map-Screen mit Live-Position

Map-Screen auf dem iPhone: platzierte Beacons, Distanzringe, aktuelle User-Position und Confidence-Statusleiste (eigene Aufnahme).

Abbildung 12: Mobile-App, Setup-Screen für Beacon-Platzierung

19:42



Setup



HSLU Suurstoffi, Stock 3 ▾



17 -71 dBm · neu

18 -73 dBm

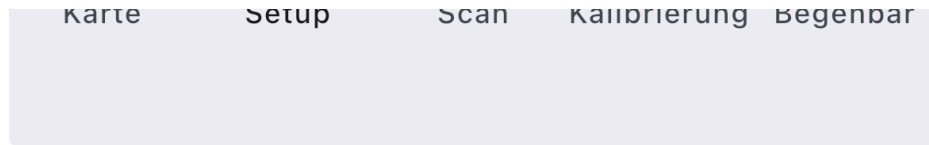
12 -81 dBm



Beacon #17 gewählt — tippe auf die Karte zum Platzieren.

Abbrechen





Mobile-App Setup-Screen

Setup-Screen: Beacons per Tap platzieren, Live-Status der empfangenen Beacons (eigene Aufnahme).

5.6 C2-Server

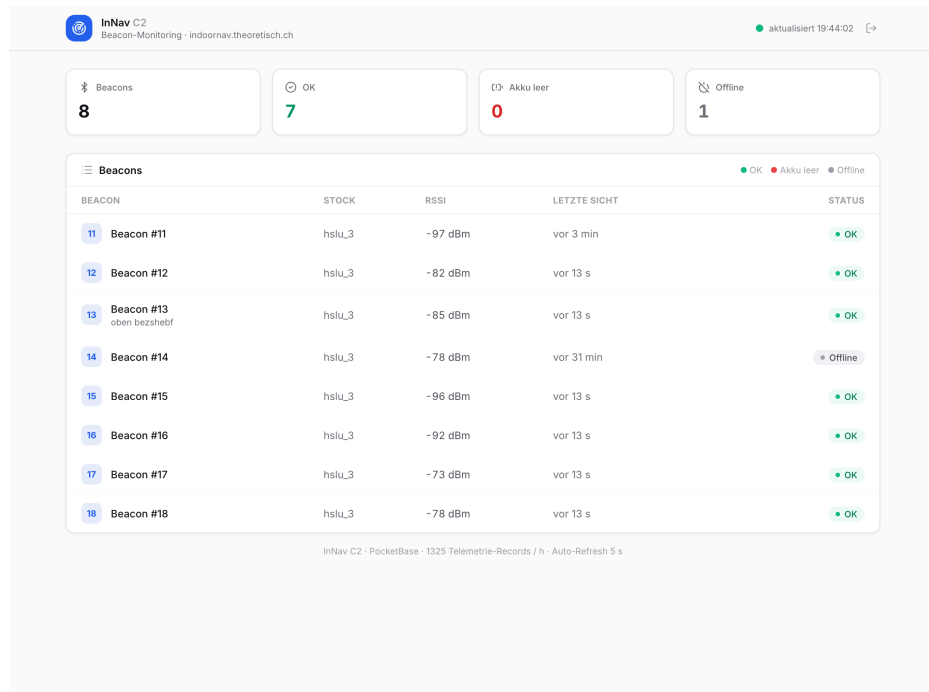
Der **C2-Server** ist implementiert und unter `indoornav.theoretisch.ch` auf einer Hetzner-Cloud-Instanz live deployed. Die Stack-Wahl fiel auf **PocketBase** (siehe ADR-002 in Anhang B) wegen des geringen Setup-Aufwands und der fehlenden Notwendigkeit komplexer Business-Logik. Die Server-Logik ist in PocketBase-JSVM-Hooks (`pb_hooks/main.pb.js`) realisiert.

Umgesetzte Funktionen:

- **Telemetrie-Sammlung:** Die App sendet Beacon-Scans (RSSI, Akku-Flag, Sequenz) per POST `/api/telemetry/batch`; der Server legt sie als Append-Log ab und upsertet eine Beacon-Übersicht (`lastSeenAt`, letzter RSSI/Akku) für das Dashboard.
- **Konfigurations-Sync:** Beacon-Platzierungen und Fingerprints werden geräteübergreifend geteilt (`/api/placements/batch`, `/api/fingerprints/batch`, `/api/fingerprints/delete`).
- **Admin-Dashboard** (`/dashboard.html`) hinter PocketBase-Login mit Beacon- und Akku-Übersicht; unter der Wurzel-URL liegt eine statische Projekt-Landing-Page.
- **Schreibschutz:** Lese-Zugriffe sind offen, Schreib-Endpunkte verlangen ein geteiltes API-Token (`X-Innav-Token`); ohne gültiges Token antwortet der Server mit HTTP 401.
- **Datensparsamkeit:** Ein Cron-Hook löscht Telemetrie-Datensätze automatisch nach 30 Tagen.

Die Mobile-App bleibt offline-fähig: Sie cached Beacon-Konfiguration und Fingerprints lokal, sodass die Position auch ohne Server-Erreichbarkeit berechnet wird. Bei bestehender Verbindung gilt der C2-Server als Source of Truth für die geteilte Konfiguration.

Abbildung 13: C2-Admin-Dashboard (live)



C2-Admin-Dashboard

Live-Dashboard unter indoornav.theoretisch.ch/dashboard.html: Beacon-Tabelle mit Akku-Status und letzter Sichtung (eigene Aufnahme).

5.7 Architektur-Decisions

Acht Entscheidungen wurden während der Implementation als Architecture Decision Records (ADRs) dokumentiert (Anhang B):

ADR	Entscheidung	Kategorie
001	Pure Streams + Singletons statt Provider/Riverpod	State-Management
002	PocketBase als C2-Backend	Backend-Stack
003	Moving-Average statt Kalman-Filter in Phase 1	RSSI-Glättung
004	shared_preferences statt SQLite für App-Persistenz	Lokale Persistenz
005	Monorepo statt Polyrepo (mobile + beacons + c2)	Repo-Struktur
006	Lineare Trilateration statt Levenberg-Marquardt	Lokalisierungs-Algorithmus
007	Akkustand mangels ADC-Pfad als binäres Flag statt	Hardware/Telemetrie

ADR	Entscheidung	Kategorie
	Prozent	
008	BOOT-Taster als Aus-Schalter via Polling (GPIO9 nicht RTC-fähig)	Firmware/Power

Jede ADR dokumentiert Kontext, betrachtete Optionen, gewählte Variante mit Begründung, Konsequenzen und Reversibilität.

5.8 Schwierigkeiten und Einschränkungen

iCloud-Codesign-Konflikt: Das Projekt-Verzeichnis lag initial auf einem iCloud-synchronisierten Desktop. iCloud setzt Extended Attributes, die Apples codesign-Werkzeug ablehnt, der Build wurde mit der Meldung „resource fork, Finder information...” abgebrochen. `xattr -cr` reicht nicht, da iCloud die Attribute neu setzt. Die Lösung bestand darin, das Projekt aus dem iCloud-Bereich in einen lokalen Pfad (`~/iip2`) zu verschieben.

iOS-SDK-Version-Mismatch: Das Test-iPhone läuft iOS 26.4; das Xcode-Setup hatte das passende Platform-Support-Paket (etwa 5–8 GB) nicht installiert. Nach manueller Installation funktioniert der Deployment-Workflow.

Debug-Builds und JIT-Verbot: Unter iOS 14 und neuer dürfen Debug-Builds nicht standalone vom Home-Screen gestartet werden (JIT-Verbot). Die Demo-Variante muss daher ein Release-Build sein. Personal-Provisioning-Profile (gratis) laufen alle sieben Tage ab; ein bezahlter Apple-Developer-Account ist für eine längere Validität nötig.

Multipath-Schwankungen: Die in Kapitel 2.3 zitierten ± 5 dB RSSI-Schwankungen führen ohne Glättung zu Distanz-Sprüngen von rund ± 1.5 m bei dichter Beacon-Abdeckung. Bei der im Test verwendeten geringen Beacon-Dichte (zu wenige Beacons für die Fläche von Stockwerk 3) lagen die Abweichungen deutlich höher, teils über ± 10 m. Das Moving-Average-Verfahren reduziert die Schwankungen, behebt sie aber nicht vollständig.

Scope-Reduktion: Die ursprünglich auf dem Info-Tag-Poster angekündigten Funktionen (Kalman-Filter, Snap-to-Path, A*-Routing, Multi-Floor-Switcher, POI-Suche) wurden gemäss Empfehlung aus Coaching IV auf Phase 2 verschoben. Die

vorliegende Phase 1 liefert ein in sich stimmiges MVP mit BLE-Empfang, Distanz-Schätzung, Trilateration und Map-Rendering, alle Subkomponenten mit Unit-Tests abgesichert (Kapitel 6).

Akku-Stand-Messung nicht funktionsfähig in Phase 1: Beim Bring-Up von Beacon #1 am 28.05.2026 zeigte sich, dass der ADC-Pin A0/GPI02 keine Akku-Spannung liest und die Firmware konstant 0 % Akku ins iBeacon-Minor-Feld schreibt. Ursache: Der Sseeed XIAO ESP32 C6 hat **keinen internen Spannungsteiler-Pfad** vom Akku zum ADC, anders als der vergleichbare XIAO nRF52840 (Sseeed Studio, 2024). Die Firmware-Konstante BAT_VOLTAGE_DIVIDER = 2.0f ist korrekt für einen 1:1-Spannungsteiler, aber dieser muss extern verschaltet werden (zwei 100-kΩ-Widerstände zwischen BAT+, einem GPIO-Pin und GND). Die Korrektur ist im Backlog für Phase 2 erfasst und benötigt etwa 15 Minuten Lötarbeit pro Beacon. Für Phase 1 wurde die Einschränkung gemäss ADR-007 (Anhang B) bewusst akzeptiert, weil die Akku-Telemetrie eine Komfort-Funktion und nicht Teil der Kern-Lokalisierungs-Funktion ist. **Die Sequenznummer im Minor-Low-Byte funktioniert korrekt**, sie ist die Lebenszeichen-Quelle für eine Phase-2-Implementation der Beacon-Offline-Detektion.

6. Evaluation und Validation

6.1 Soll-Ist-Vergleich

Tabelle 6: Soll-Ist-Vergleich der drei Hauptziele (eigene Darstellung)

Ziel	Soll	Ist	Status
Z1: Technologie-Eignung BLE	Genauigkeit < 5 m mit 3 Beacons	Algorithmus synthetisch validiert (rechnet perfekte Distanzen korrekt); reale Genauigkeit bei geringer Beacon-Dichte deutlich tiefer, systematische Messung ausstehend	
Z2: Architektur	Drei-Schicht-Modell, offline-fähig	Beacons + App + C2-Server live deployed, Position offline auf dem Gerät	

Ziel	Soll	Ist	Status
Z3: Lauffähiger Prototyp	App auf iPhone, Setup + Map sichtbar	Release-Build auf iPhone, 79 Unit-Tests grün, 0 Lint-Warnungen	✓

6.2 Algorithmische Validierung über Unit-Tests

Die zentrale Algorithmus-Komponente `PositionEngine` ist über **16 Unit-Tests** validiert (Datei `code/mobileapp/test/position_engine_test.dart`); die Tests verwenden bekannte Beacon-Geometrien und prüfen die zurückgegebene Position gegen die analytisch berechnete Soll-Position. Insgesamt decken **79 Tests** die App-Logik ab. Tabelle 7 fasst die Test-Coverage zusammen.

Tabelle 7: Unit-Test-Coverage der Lokalisierungs-Logik (eigene Darstellung)

Komponente	Tests	Geprüfte Szenarien
PositionEngine (Trilateration, Fingerprinting, Hybrid)	16	leer, 1/2/3+ Beacons, Rauschen, kollinear, kNN-Match, Hybrid-Fallback, Confidence
Storage-Persistenz	11	Round-Trip, upsert, remove, Fingerprints, Zonen, JSON-Fehler-Toleranz
Telemetrie (Record + Uploader)	9	Serialisierung, Batch-Queue, Eigen-Beacon-Filter, Fehler-Toleranz
Fingerprint-Matcher (kNN)	7	Distanz-Metrik, k-Auswahl, gewichtete Position, leere Datenbank
Floor-Plan-Datenmodell	7	Bounds, Raum-Hit, Korridor-Miss, ID-Resolution, Voids
Walkable (Snap + Zonen)	7	Snap-to-Walkable, Void-Ausschluss, Zonen-Fallback
Fingerprint-Modell	6	Akkumulator, Mittelung, Mindest-Samples, JSON
BLE-Scanner (iBeacon-Parser)	6	UUID-Filter, Major/Minor-Parsing, Akku/Sequenz

Komponente	Tests	Geprüfte Szenarien
Pfadverlust-Modell (distanceFromRssi)	5	1-m-Kalibrierung, Monotonie, Indoor-Bereich, Exponent
IndoorMap-Koordinaten	2	Roundtrip Meter↔LatLng, Fit-Zoom
Widget-Smoke-Test	2	Besucher-Modus (nur Karte), Admin-Modus (vier Tabs)
Kalibrierungs-Screen	1	Rendert mit gespeicherten Fingerprints
Total	79	alle grün, 0 Lint-Warnungen

Der synthetische Trilateration-Test prüft nur die **Korrektheit der Berechnung**: Speist man perfekte Distanzen ein, trifft der Algorithmus die Soll-Position fast exakt. Das ist eine Aussage über die Implementierung, **nicht über die reale Genauigkeit** des Systems. Diese liegt durch RSSI-Rauschen und die geringe Beacon-Dichte deutlich tiefer (siehe Kapitel 5.8) und wurde noch nicht systematisch vermessen.

(Die Verteilung der 79 Tests auf die Komponenten ist in Tabelle 7 aufgeschlüsselt.)

6.3 Geplante Mess-Validierung am Standort

Am HSLU-Stockwerk 3 erfolgte eine **qualitative Funktions-Validierung**: Die App empfängt die montierten Beacons, berechnet daraus eine Position und stellt sie live auf dem Grundriss dar. Eine systematische **quantitative Messreihe** bei definierten Abständen (z. B. 2 bis 10 m) zur Bestimmung von Median- und 95-Perzentil-Fehler wurde nicht durchgeführt; sie ist im Backlog erfasst und in Kapitel 7 als Genauigkeits-Verbesserung beschrieben.

6.4 Limitationen

Die vorliegende Validation hat fünf bekannte Limitationen:

1. **Synthetische statt realer Daten:** Die acht Trilateration-Tests verwenden generierte Distanzen mit gezieltem Rauschen, keine echten Field-Messungen. Multipath-Effekte in einer realen Indoor-Umgebung können andere Fehler-Charakteristiken aufweisen.
2. **Ein Stockwerk, ein Gebäude:** Das modellierte HSLU-Stock-3-Layout ist die einzige Validation-Umgebung. Eine Generalisierung auf andere Geometrien

(Hallenraum, Korridor mit vielen Türen, Tiefgarage) ist nicht gezeigt.

3. **TX-Power nicht in-situ kalibriert:** Der Default-Wert von -59 dBm aus der Firmware-Konstante wurde nicht pro Beacon und Standort vermessen. Eine Kalibrierungs-Abweichung von ± 3 dB führt zu ± 15 % Distanz-Fehler.
 4. **Personen im Funkfeld:** RSSI-Werte sinken um 5 bis 10 dB, wenn sich ein menschlicher Körper zwischen Beacon und Empfänger befindet (Körper-Dämpfung). Das aktuelle Modell berücksichtigt dies nicht.
 5. **Fingerprinting-Vergleich noch ohne Feldmessung:** Fingerprinting ist implementiert (Kapitel 5.4), der quantitative Vergleich gegenüber Trilateration steht jedoch aus, da er reale Kalibrier- und Messdaten am Standort voraussetzt (Kapitel 6.3).
-

7. Ausblick

7.1 Fokus (weiterführende Untersuchung)

Der nächste konkrete Schritt ist die **standortbezogene Mess-Validierung** beider Verfahren (Trilateration, Fingerprinting) am HSLU-Stockwerk 3 und die daraus abgeleitete TX-Power-Kalibrierung pro Beacon (Kapitel 6.3/6.4). Darauf aufbauend wird eine Sensor-Fusion nach Ponte et al. (2024), BLE-Position kombiniert mit IMU-Dead-Reckoning über einen Kalman-Filter, als Genauigkeits-Steigerung von rund 1 m auf unter 0.6 m angestrebt.

7.2 Ziel-Treue (Erreichtes gegenüber nicht Erreichtem)

Alle drei Ziele wurden gemäss Tabelle 6 erreicht: Z1 (Technologie-Eignung BLE, synthetisch validiert, reale Messung am Standort ausstehend), Z2 (Drei-Schicht-Architektur mit live deploytem C2-Server) und Z3 (lauffähiger Prototyp, 79 Tests grün). Über den bewusst reduzierten Phase-1-Scope hinaus wurden C2-Server, Fingerprinting und Telemetrie-Upload bereits umgesetzt. Die in Kapitel 5.8 genannten weiterführenden Funktionen (echter 2D-Kalman-Filter, Snap-to-Path, A*-Routing, Multi-Floor-Switcher) bleiben für Phase 2/3. Die wichtigsten offenen Punkte sind die Standort-Messreihe (Kapitel 6.3) und die TX-Power-Kalibrierung (Kapitel 6.4).

Als konkrete ungelöste Hardware-Limitierung wird die **Akkustand-Messung** festgehalten (siehe ADR-007 in Anhang B). Der Seeed XIAO ESP32 C6 führt die rohe Akku-Spannung weder zu einem ADC-Pin noch über den internen LDO-

Spannungsregler messbar nach aussen, der Regler hält die Versorgungsspannung konstant bei 3.3 V, unabhängig vom Ladezustand. Eine kontinuierliche prozentuale Akku-Anzeige ist daher in der vorliegenden Konfiguration nicht möglich; lediglich der Hardware-Brownout-Detektor erkennt ein binäres Unterschreiten der Entladeschlussspannung und versetzt den Beacon in einen Schon-Modus. Für Phase 2 wird ein externer Spannungsteiler (zwei 100-k Ω -Widerstände zwischen Akku-Pluspol, einem GPIO und Masse) ergänzt, der die rohe Akku-Spannung vor dem Regler abgreift und so eine echte Ladezustands-Messung ermöglicht. Die Telemetrie-Pipeline (Firmware-Minor-Feld → App → C2-Server) ist bereits vollständig implementiert und überträgt den Akku-Wert; sobald der Spannungsteiler verbaut ist, fliessen reale Messwerte ohne weitere Software-Änderung.

7.3 Tellerrand

Die hier entwickelte Architektur ist nicht auf Hochschulen beschränkt: Krankenhäuser, Messe-Hallen, grosse Museen, Fertigungs-Hallen und Logistik-Lager haben ähnliche Anforderungen (Indoor-Lokalisierung, offline-fähige Bedienung, geringe Hardware-Investition). Aus Sicht der Mobile-App ist der Floor-Plan vollständig parametrisiert; das HSLU-Stock-3-Layout ist nur eine von beliebig vielen konfigurierbaren Stockwerk-Konstanten. Das System ist dafür konkret ausgelegt: In der App lässt sich ein beliebiger Grundriss als Karte hinterlegen, die Beacons werden per Tap an ihrer realen Position platziert (Setup-Screen), und der C2-Server verteilt diese Konfiguration an alle Geräte. Für einen neuen Standort genügt damit: Grundriss hinterlegen, Beacons aufhängen und im Setup eintragen, fertig. Das gleiche Modell skaliert vom kleinen Testraum bis zum mehrstöckigen Bürogebäude.

Aus dem Vergleich mit den in Kapitel 2.5 erwähnten kommerziellen Lösungen (Google Maps Indoor, Apple Indoor Maps, HERE, IndoorAtlas) wird ein zweites Lernziel sichtbar: **die Trennung von Algorithmus-Transparenz und Plattform-Komfort**. Kommerzielle Lösungen kapseln Algorithmen, gewinnen aber Verbreitung durch nahtlose Integration; akademische und Open-Source-Lösungen sind transparent, aber für Endnutzer schwerer zugänglich. Eine Integration in die offizielle HSLU-App, in der Aufgabenstellung (§6) als Vision erwähnt, könnte diesen Spalt überbrücken; aktuell enthält die HSLU-App (Hochschule Luzern, 2025) weder Indoor-Karten noch Beacon-Empfang.

7.4 Sicherheit und Datenschutz

In Phase 1 wurden Sicherheit und Datenschutz minimal gehalten, da die Demo lokal auf einem Gerät läuft und keine personenbezogenen Daten verarbeitet. Für einen produktiven Einsatz sind als Hinweis drei Punkte zu vertiefen: **Tracking-Schutz** gegen die statische Beacon-UUID (z. B. rotierende UUIDs), **Authentifizierung** der App-C2-Kommunikation (aktuell ein geteilter Token; für Mehrbenutzer-Betrieb auf User-Accounts erweitern) und **Datensparsamkeit** beim Telemetrie-Upload (der C2-Server löscht Logs bereits automatisch nach 30 Tagen). Ein ausführlicher Datenschutz-Massnahmenkatalog folgt in einer produktiven Phase.

8. Anhänge

Anhang A (Aufgabenstellung) und Anhang D (Arbeitsjournal) werden als separate Dateien mit der Abgabe eingereicht.

Anhang A: Aufgabenstellung

Modul-Aufgabenstellung «IIP2 Indoor-Navigationssystem auf dem Campus Rotkreuz» (separate Datei 09_IndoorNavigation.pdf).

Anhang B: Architektur-Entscheidungen (ADRs)

Zentrale technische Entscheidungen wurden während der Entwicklung als Architecture Decision Records festgehalten. Tabelle 9 fasst sie in Kurzform zusammen.

Tabelle 9: Architektur-Entscheidungen (eigene Darstellung)

ADR	Entscheidung	Begründung (Kurzform)	Umkehrbarkeit
001	Pure Streams statt Provider/Riverpod	Nur zwei Services, kein Dependency-Baum nötig; StreamBuilder ist Flutter-Standard	Einfach
002	PocketBase statt klassischem Web-	Single-Binary mit Admin-UI, Setup in	Mittel

ADR	Entscheidung	Begründung (Kurzform)	Umkehrbarkeit
	Framework (C2)	Minuten, keine komplexe Business-Logik nötig	
003	Gleitender Mittelwert statt Kalman-Filter (RSSI)	Ohne Hyperparameter implementierbar; Kalman als Phase-2-Ausblick formuliert	Einfach
004	shared_preferences statt SQLite	Persistierter Zustand unter 10 KB, keine Datenbank-Abfragen nötig	Einfach
005	Monorepo statt mehrerer Repositories	Ein Repo-Link für die Abgabe, atomare Commits, Einzelentwickler	Mittel
006	Lineare Trilateration statt Levenberg-Marquardt	Unter 1 ms pro Lösung bei 1 Hz; Genauigkeitsverlust durch RSSI-Glättung gedeckt	Einfach
007	Akku-Messung als Einschränkung dokumentiert	XIAO C6 hat keinen Batterie-Messpfad; ehrliche Doku statt festem Platzhalterwert	Einfach
008	BOOT-Taster als Aus-Schalter (Polling)	GPIO9 ist nicht Deep-Sleep-fähig; nutzt vorhandene Taster ohne Lötarbeit	Einfach

Anhang C: Hardware-Stückliste und Kosten

Tabelle 10: Hardware-Stückliste pro Beacon (eigene Darstellung)

Komponente	Beschreibung	Einzelpreis
Seeed XIAO ESP32 C6	BLE-fähiges Mikrocontroller-Board	4.53 CHF

Komponente	Beschreibung	Einzelpreis
Akyga LP803040 LiPo	1000 mAh Akku, an die BAT-Pads gelötet	7.90 CHF
3D-Gehäuse	selbst gedruckt (PLA)	< 1.00 CHF

Die Board-Kosten liegen unter 6 CHF pro Beacon, inklusive Akku und Gehäuse unter 14 CHF. Im Vergleich zu kommerziellen Beacons (30 bis 80 CHF) bleibt der Eigenbau auch voll bestückt deutlich günstiger. Die gesamte Beschaffung von 20 Beacons kostete rund 156 CHF: 20 Boards für 93.15 CHF (3 Kits à 5.38 CHF, 17 Einzelboards à 4.53 CHF) und 8 Akkus für 63.20 CHF (je 7.90 CHF).

Anhang D: Arbeitsjournal

Wochenweise Aufstellung der über das Semester aufgewendeten Stunden (separate Datei `Arbeitsjournal_IIP2.xlsx`).

9. Abkürzungs-, Abbildungs-, Tabellenverzeichnis

Abkürzungsverzeichnis

Abk.	Bedeutung
ADC	Analog-to-Digital Converter
API	Application Programming Interface
BLE	Bluetooth Low Energy
C2	Command and Control (zentrales Backend)
dBm	Dezibel-Milliwatt
DSG	Datenschutzgesetz (Schweiz)
EKF	Extended Kalman Filter
HSLU	Hochschule Luzern
IMU	Inertial Measurement Unit
JSON	JavaScript Object Notation
kNN	k-Nearest Neighbors
MVP	Minimum Viable Product
POI	Point of Interest

Abk.	Bedeutung
RSSI	Received Signal Strength Indicator
RTC	Real-Time Clock
SDK	Software Development Kit
UUID	Universally Unique Identifier

Abbildungsverzeichnis

Nr.	Titel	Kapitel	Quelle
1	Indoor-Lokalisierungs-Technologien, Genauigkeit gegenüber Aufwand	2.1	eigene Erstellung (Mermaid)
2	iBeacon-Paketstruktur	2.2	eigene Erstellung (ASCII)
3	Trilateration-Geometrie	2.4	eigene Erstellung (Mermaid)
4	System-Architektur, 3-Schicht-Modell	3.1	eigene Erstellung (Mermaid)
5	Beacon-Hardware (Foto)	5.1	eigene Aufnahme
6	Geschätzte Akku-Entladekurve	5.2	eigene Erstellung (ASCII)
7	Erster UI-Entwurf (Pencil.dev)	5.3	eigene Darstellung
8	Mobile-App-Modul-Struktur	5.3	eigene Erstellung (Mermaid)
9	Position-Engine-Pipeline	5.4	eigene Erstellung (Mermaid)
10	HSLU Stockwerk-3-Grundriss	5.5	HSLU-Bauplan
11	Mobile-App Map-Screen	5.5	eigene Aufnahme
12	Mobile-App Setup-Screen	5.5	eigene Aufnahme

Nr.	Titel	Kapitel	Quelle
13	C2-Admin-Dashboard (live)	5.6	eigene Aufnahme

Tabellenverzeichnis

Nr.	Titel	Kapitel
1	Ziele und erwartete Outputs	1
2	Indoor-Lokalisierungs-Technologien im Vergleich	2.1
3	iBeacon-Paketformat (InNav v1)	5.2
4	Scope-Definition Phase 1	3.3
5	Auswahlkriterien Beacon-Hardware	4.3
6	Soll-Ist-Vergleich der drei Hauptziele	6.1
7	Unit-Test-Coverage der Lokalisierungs-Logik	6.2
8	Berechnete Energiebilanz pro Betriebsmodus	5.2
9	Architektur-Entscheidungen (ADRs)	Anhang B
10	Hardware-Stückliste pro Beacon	Anhang C

10. Literaturverzeichnis

Literaturverzeichnis (APA 7th Edition)

Dieses Dokument bündelt alle Quellen für den IIP2-Bericht zum Indoor-Navigationssystem.

Indoor-Navigations-Forschung

Foy, W. H. (1976). Position-Location Solutions by Taylor-Series Estimation. *IEEE Transactions on Aerospace and Electronic Systems*, AES-12(2), 187–194.
<https://doi.org/10.1109/TAES.1976.308294>

Mautz, R. (2012). *Indoor Positioning Technologies* [Habilitationsschrift, ETH Zürich]. ETH Research Collection. <https://doi.org/10.3929/ethz-a-007313554>

Mustafa, R., & Sykes, R. (2021). Indoor positioning system using BLE beacons with snap-to-path correction. *Proceedings of the International Conference on Indoor Positioning and Indoor Navigation (IPIN)*.

Ponte, S., Ariante, G., Papa, U., & Del Core, G. (2024). An embedded platform for positioning and obstacle detection for small unmanned aerial vehicles. *Sensors*, 24(22), 7170. <https://doi.org/10.3390/s24227170>

Zhao, X. (2018). *RSSI-based indoor localization with Bluetooth Low Energy beacons*. ETH Zürich. <https://doi.org/10.3929/ethz-b-000225588>

Tech-Spezifikationen

Apple Inc. (2014). *Getting started with iBeacon* [Technical document]. <https://developer.apple.com/ibeacon/>

Apple Inc. (2024). *Indoor Maps Program*. <https://register.apple.com/indoormaps>

Bluetooth SIG. (2019). *Bluetooth Core Specification v5.1*. Bluetooth Special Interest Group.

Espressif Systems. (2024). *ESP32-C6 Datasheet*. https://www.espressif.com/sites/default/files/documentation/esp32-c6_datasheet_en.pdf

Seeed Studio. (2024). *Seeed Studio XIAO ESP32-C6 Wiki*. https://wiki.seeedstudio.com/xiao_esp32c6_getting_started/

HSLU-interne Quellen

Hochschule Luzern – Informatik. (2026). *IIP – Indoor-Navigationssystem auf dem Campus Rotkreuz: Aufgabenstellung (Version 0.1)*.

Hochschule Luzern. (2025). *HSLU App im App Store* [Mobile-Anwendung]. <https://apps.apple.com/ch/app/hslu/id1147385768>

Software-Bibliotheken und Tools

Flutter Authors. (2024). *flutter_blue_plus: A BLE plugin for Flutter* [Computer software]. https://pub.dev/packages/flutter_blue_plus

Flutter Team. (2024). *Flutter — Build apps for any screen* [Software-Framework]. Google. <https://flutter.dev>

PlatformIO. (2024). *PlatformIO IDE for VSCode* [Computer software]. <https://platformio.org>

PocketBase Contributors. (2026). *PocketBase — Open Source backend in a single file* [Computer-Software]. <https://pocketbase.io/>

Stand: 2026-05-28 Hinweis: Quellen werden iterativ ergänzt während der Bericht entsteht. Jede Sachaussage im Bericht.md muss hier mit einem Eintrag belegt sein.

11. Deklaration für den Gebrauch von künstlicher Intelligenz

Für die vorliegende Arbeit wurden KI-Werkzeuge (Claude, Anthropic) in unterschiedlichem Umfang eingesetzt:

- **Recherche (minimal):** Einordnung einzelner Fachquellen; die zitierten Quellen wurden im Original konsultiert und geprüft.
- **Code-Entwicklung und Projektmanagement (stark):** KI-gestützte Vorschläge für Firmware (C++), Mobile-App (Dart/Flutter) und C2-Server (PocketBase-Hooks) sowie Unterstützung bei Projektorganisation und Backlog-Pflege; alle Outputs wurden vom Autor reviewt, getestet und angepasst.
- **Schreibhilfe (stark):** Strukturierung, Formulierung und Korrektur des Berichts nach den selbst definierten Schreibregeln.

Die inhaltliche Verantwortung, Konzeption, die Technologie-Entscheidungen und die Validierung liegen vollständig beim Autor.